



PRACTICAL - 7

AIM : To Implementation of making a change problem using dynamic programming

Method 1 : total no of ways

Algorithm :

Input: coin values and amount w
Output: total no. of ways
Step 1: start
Step 2: Read n, coin values and amount w
Step 3: Set $A[i][0] = 1$ For $i = 0$ to n
Step 4: Set $A[0][j] = 0$ For $j = 1$ to w
Step 5: Repeat For $i = 1$ to n
Step 6: Repeat for $j = 1$ to w
Step 7: If $\text{coin}[i] > j$, set
 $A[i][j] = A[i-1][j]$
Step 8: otherwise, set
 $A[i][j] = A[i-1][j] + A[i][j - \text{coin}[i]]$
Step 9: print $A[n][w]$
Step 10: Stop

code :

```
#include <iostream>
using namespace std;
```

```
class ChangeMaking {
private:
    int coins[10];
    int n;
```

```
int w;  
int A[10][100];
```

```
public:
```

```
void getInput() {  
    cout << "Enter number of coin types : ";  
    cin >> n;  
  
    cout << "Enter coin values : ";  
    for (int i = 1; i <= n; i++) {  
        cin >> coins[i];  
    }  
  
    cout << "Enter amount : ";  
    cin >> w;  
}  
  
void solve() {  
  
    // Base case: one way to make amount 0  
    for (int i = 0; i <= n; i++) {  
        A[i][0] = 1;  
    }  
  
    // Base case: no way to make positive amount with 0 coins  
    for (int j = 1; j <= w; j++) {  
        A[0][j] = 0;  
    }  
  
    // Fill DP table  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= w; j++) {  
  
            if (coins[i] > j) {  
                // Current coin cannot be used  
                A[i][j] = A[i - 1][j];  
            }  
            else {  
                // Do not use coin + Use coin  
                A[i][j] = A[i - 1][j]  
                    + A[i][j - coins[i]];  
            }  
        }  
    }  
}
```

```
    }  
  }  
}  
  
void printTable() {  
  
    cout << "\nDP Table:\n";  
  
    cout << "Coin\t";  
  
    for (int j = 0; j <= w; j++) {  
        cout << j << "\t";  
    }  
  
    cout << endl;  
  
    for (int i = 0; i <= n; i++) {  
  
        if (i == 0)  
            cout << "0\t";  
        else  
            cout << coins[i] << "\t";  
  
        for (int j = 0; j <= w; j++) {  
            cout << A[i][j] << "\t";  
        }  
  
        cout << endl;  
    }  
}  
  
void showResult() {  
    cout << "\nTotal Number of Ways = "  
        << A[n][w] << endl;  
}  
};  
  
int main() {  
  
    ChangeMaking c;
```

```
c.getInput();
c.solve();
c.printTable();
c.showResult();

return 0;
}
```

Output :

```
Enter number of coin types : 3
Enter coin values : 2
1
3
Enter amount : 10

DP Table:
Coin  0    1    2    3    4    5    6    7    8    9    10
0      1    0    0    0    0    0    0    0    0    0
2      1    0    1    0    1    0    1    0    1    0    1
1      1    1    2    2    3    3    4    4    5    5    6
3      1    1    2    3    4    5    7    8    10   12   14

Total Number of Ways = 14
```

Time complexity :

Best case :

```
for (int j=0; j<=w; j++)
    cout<<j<<" ";
cout<<endl;

for (int i=0; i<=n; i++) {
    if (i==0)
        cout<<"0 ";
    else
        cout<<coins[i]<<" ";
    for (int j=0; j<=w; j++)
        cout<<A[i][j]<<" ";
    cout<<endl;
}

Time complexity = O(nw)
```

Average case :



```
for (int j=0; j<=w; j++)  
    cout<<j<<" ";  
cout<<endl;  
for (int i=0; i<=n; i++) {  
    if (i==0)  
        cout<<"0 ";  
    else  
        cout<<coins[i]<<" ";  
    for (int j=0; j<=w; j++)  
        cout<<A[i][j]<<" ";  
    cout<<endl;  
}
```

Time complexity = $O(nw)$

Worst case :

```
for (int j=0; j<=w; j++)  
    cout<<j<<" ";  
cout<<endl;  
for (int i=0; i<=n; i++) {  
    if (i==0)  
        cout<<"0 ";  
    else  
        cout<<coins[i]<<" ";  
    for (int j=0; j<=w; j++)  
        cout<<A[i][j]<<" ";  
    cout<<endl;  
}
```

Time complexity = $O(nw)$



Method 2 : minimum number of ways

Algorithm :

minimum no. of coins.
Input: coin values and amount w
Output: minimum no of ways
Step 1: Start
Step 2: Read n, coin values and amount w
Step 3: Set $A[i][0] = 0$ for $i = 0$ to n
Step 4: Set $A[0][j] = 999$ for $j = 1$ to w .
Step 5: Repeat for $i = 1$ to n
Step 6: Repeat for $j = 1$ to w
Step 7: if $\text{coin}[i] > j$, set
 $A[i][j] = A[i-1][j]$,
Step 8: otherwise, set
 $A[i][j] = \min(A[i-1][j], 1 + A[i][j - \text{coin}[i]])$
Step 9: print $A[n][w]$
Step 10: stop

Code :

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```
class ChangeMaking {
private:
    int coins[10];
    int n, w;
    int A[100][100];
```

public:

```
void getInput() {
    cout << "Enter number of coin types: ";
    cin >> n;

    cout << "Enter coin values: ";
    for (int i = 1; i <= n; i++) {
        cin >> coins[i];
    }

    cout << "Enter amount: ";
    cin >> w;
}

void solve() {

    // Base case: 0 coins are required to make amount 0
    for (int i = 0; i <= n; i++) {
        A[i][0] = 0;
    }

    // If no coins are available, amount cannot be formed
    for (int j = 1; j <= w; j++) {
        A[0][j] = 999;
    }

    // Dynamic Programming
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {

            if (coins[i] > j) {
                A[i][j] = A[i - 1][j];
            }
            else {
                A[i][j] = min(
                    A[i - 1][j],
                    1 + A[i][j - coins[i]]
                );
            }
        }
    }
}
```



```
}

void printTable() {

    cout << "\nDP Table:\n";

    cout << "    ";
    for (int j = 0; j <= w; j++) {
        cout << j << " ";
    }
    cout << endl;

    for (int i = 0; i <= n; i++) {

        cout << "i=" << i << " ";

        for (int j = 0; j <= w; j++) {

            if (A[i][j] == 999)
                cout << "0 ";
            else
                cout << A[i][j] << " ";
        }

        cout << endl;
    }
}

void showResult() {

    if (A[n][w] == 999) {
        cout << "\nAmount cannot be formed.";
    }
    else {
        cout << "\nMinimum number of coins = "
            << A[n][w];
    }
}

};

int main() {
```

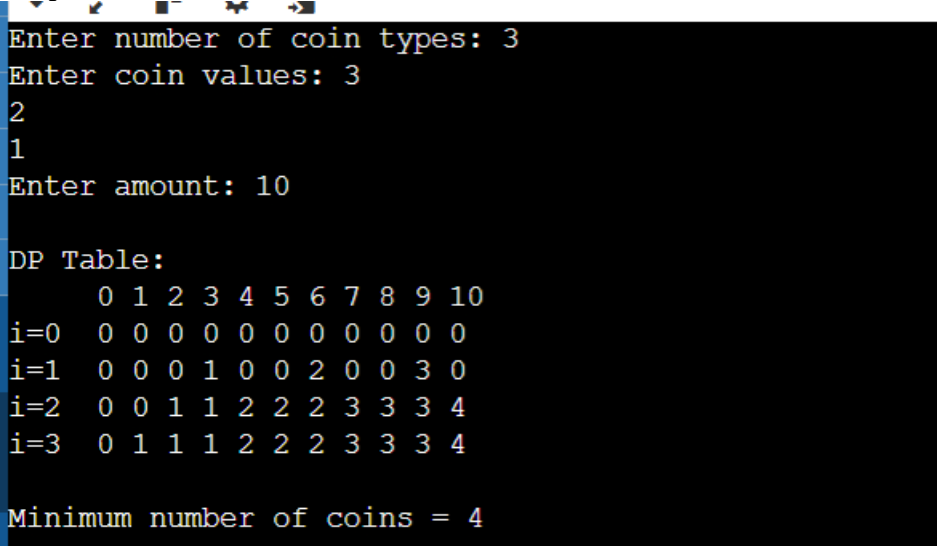
ChangeMaking c;

```
c.getInput();  
c.solve();  
c.printTable();  
c.showResult();
```

```
return 0;
```

```
}
```

Output :



```
Enter number of coin types: 3  
Enter coin values: 3  
2  
1  
Enter amount: 10  
  
DP Table:  
   0 1 2 3 4 5 6 7 8 9 10  
i=0 0 0 0 0 0 0 0 0 0 0 0  
i=1 0 0 0 1 0 0 2 0 0 3 0  
i=2 0 0 1 1 2 2 2 3 3 3 4  
i=3 0 1 1 1 2 2 2 3 3 3 4  
  
Minimum number of coins = 4
```

Time complexity :

Best case :



```
for(int j=0; j<=w; j++) _____ w
    cout<< j<<" ";
cout<< endl;
for (int i=0; i<=n; i++) { _____ n
    cout<<"i="<<i<<" ";
    for (int j=0; j<=w; j++) { _____ w
        if(A[i][j]==999)
            cout<<"0";
        else
            cout<< A[i][j]<<" ";
    }
    cout<< endl; }
```

time complexity

$$T(n,w) = w \times n \times w \times O(1) \\ = 2nw \\ = O(nw)$$

Average case :

```
for(int j=0; j<=w; j++) _____ w
    cout<< j<<" ";
cout<< endl;
for (int i=0; i<=n; i++) { _____ n
    cout<<"i="<<i<<" ";
    for (int j=0; j<=w; j++) { _____ w
        if(A[i][j]==999)
            cout<<"0";
        else
            cout<< A[i][j]<<" ";
    }
    cout<< endl; }
```

time complexity

$$T(n,w) = w \times n \times w \times O(1) \\ = 2nw \\ = O(nw)$$

Worst case :



```
for(int j=0; j<=w; j++) _____ w
    cout<< j<<" ";
cout<< endl;
for(int i=0; i<=n; i++) _____ n
    cout<< "i="<<i<<" ";
    for(int j=0; j<=w; j++) _____ w
        if(A[i][j]==999)
            cout<<"0";
        else
            cout<< A[i][j]<<" ";
    cout<< endl;}
```

time complexity
 $T(n,w) = w \times n \times O(1)$
 $= n \times w$
 $= O(nw)$

Conclusion :

The Making Change Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the minimum number of coins required for a given amount with a time complexity of $O(nW)$.